

Nathan Hines HCI 584 Project Ideas/Sketches

Project 1

Description

A web app targeted at LA-area beachgoers that pulls real-time and forecast weather, UV, and surf/wave data for a selected local beach and returns a simple, human-readable verdict: **Go, Maybe, or Stay Home**. Rather than making the user interpret raw weather data themselves, the app does the reasoning. For example, if water temp is cold, swell is high, UV is extreme, or there's rain in the forecast, it factors all of that in and explains *why* it made its call.

The user selects a beach (e.g. Santa Monica, Venice, Zuma, Malibu) from a dropdown and picks a date (today or up to ~7 days out) along with possible activities like sunbathing, swimming, surfing etc. The app then displays the verdict prominently along with a summary card showing the key conditions: temperature, wind speed, wave height, UV index, and chance of rain. A simple visual dashboard (colored indicators or simple charts) makes it easy to scan. The tone should be casual and fun.

Use Case / Vignette

It's Thursday evening and Nathan is trying to decide whether to head to the beach Saturday morning. He opens the app, selects "Venice Beach" from the dropdown and Sunbathing and Swimming as activities, and picks Saturday for day. The app returns a "Maybe" verdict because the weather looks great (82°F, sunny) but wave height is flagged as higher than ideal for casual swimming. The summary card shows a wind speed of 18 mph and a swell of 5ft. Nathan decides to go anyway but plans to skip swimming and just hang.

Biggest Implementation Challenge

Getting reliable, free surf/wave data is probably the trickiest part. Can use Open-Meteo handles weather and UV well for free, but wave/swell data for specific LA beaches may require a second API (Did some digging with AI and Stormglass has a free tier but it's rate-limited). Possible fallback would be to use Open-Meteo's marine forecast endpoints which do include wave height. I can realistically build the API calls to Open-Meteo (weather + UV + marine), may need help doing multi-API and combining that. Frontend would be Flask web app (or something similar).

Project 2

Description

A data dashboard that lets users compare the cost of living between two cities side by side breaking it down by category (rent, groceries, transportation, dining out, utilities) rather than just showing a single index number. The goal is to make relocation decisions, salary negotiation, or general curiosity much more informed and visual.

The user selects two cities from a searchable dropdown and the app pulls structured cost of living data and renders a comparison. Ideas being a grouped bar chart by category, a summary table, and a headline "City B is X% more/less expensive than City A overall." The user can also enter their current salary and the app will calculate what equivalent purchasing power would look like in the comparison city (a "salary translator"). This makes the output personally meaningful rather than abstract.

The data source would be the Teleport API (free, no key required), which provides quality of life and cost of living scores broken down by category for hundreds of cities worldwide.

Use Case / Vignette

Nathan is a remote worker currently living in Austin, TX making \$95,000/year. He's been offered a role in Denver and wants to know if the salary is comparable. He opens the app, types "Austin" and "Denver" into the city fields, and hits Compare. The app shows a side-by-side bar chart, Denver is about 12% more expensive overall, with rent being the biggest gap. He enters his Austin salary and the app tells him he'd need ~\$106,500 in Denver to maintain the same lifestyle. He shares the results link with his partner.

Biggest Implementation Challenge

The Teleport API data is city-level and uses its own city slug system, so building a clean city search/autocomplete that maps to valid Teleport slugs is the most unclear part to me. Some smaller or non-US cities may have incomplete data. A fallback would be to use a pre-downloaded CSV dataset (e.g. from Numbeo or Kaggle) which trades live data for reliability and simplicity. Then a Flask web app (or something similar) for frontend.

Project 3

Description

A personal fitness tracking web app (Flask or something similar maybe Streamlit – recommended by AI) where the user logs their workouts over time and can visualize their progress. The focus is on strength training where the user logs an exercise name, sets, reps, and weight for a given date, and the app stores this locally in a CSV file. Over time, the app becomes a personal training history that can be queried and charted.

The real value is in the visualization layer so the user can select any exercise (e.g. Bench Press) and see a line chart of their max weight over time, total volume per session, or average reps. A summary dashboard shows recent activity, current streaks, and personal records (PRs) per exercise which automatically detected from the log. There's no external API needed so the app is entirely self-contained, which keeps it simple and private.

Use Case / Vignette

Nathan has been lifting for three months but has just been keeping notes in his phone's Notes app. He opens the Workout Logger, clicks "Log Workout," selects today's date, and adds three exercises: Squat (4 sets, 185 lbs), Bench Press (3 sets, 135 lbs), and Deadlift (3 sets, 225 lbs). He hits Save. Later that week, he goes to the Progress tab, selects "Squat," and sees a line chart showing his max weight per session going from 135 lbs to 185 lbs over 10 sessions.

Biggest Implementation Challenge

The trickiest design question for me is the data model and how to structure a CSV (or SQLite table - recommended by AI) that cleanly handles multiple exercises per session, multiple sets per exercise, and still queries well for charting.